

1. What are advantages of DBMS over traditional file-based systems?

Reduced data redundancy. In a file-based system, data can be stored in multiple places, which can lead to data redundancy. This can be a problem because it can make data inconsistent and difficult to manage.

Improved data integrity. DBMSs enforce data integrity constraints, which ensures that the data in the database is accurate and consistent. This is not possible with file-based systems, where each application is responsible for managing its own data.

Enhanced data security. DBMSs provide a variety of security features, such as user authentication, access control, and encryption. This helps to protect data from unauthorized access, modification, or deletion. File based systems do not have the same level of security features.

Efficient data access. DBMSs use indexes and other techniques to improve the performance of data access. This can be a major advantage for applications that need to access large amounts of data quickly. File based systems do not have the same level of performance.

Scalability. DBMSs are scalable, which means that they can be easily expanded to accommodate more data and users. File based systems are not as scalable, which can be a problem for growing organizations.

- **Data searching:** For every search operation performed on the file system, a different application program has to be written. While DBMS provides inbuilt searching operations. The user only has to write a small query to retrieve data from the database.
- **Backup:** It creates a backup subsystem to restore the data if required.
- **Interfaces:** It provides different multiple user interfaces like graphical user interface and application program interface.
- **Easy Maintenance:** It is easily maintainable due to its centralized nature.

2. What are primary and foreign keys?

A **primary key** is a column or group of columns in a table that uniquely identifies each row in the table. A primary key cannot have duplicate values, and it cannot be null.

A **foreign key** is a column or group of columns in a table that references the primary key of another table. A foreign key ensures that the values in the foreign key columns exist in the referenced table's primary key columns.

3. What is the difference between primary key and unique constraints?

Primary keys are the only unique constraint that can be used to uniquely identify a row in a table. This means that a table can only have one primary key. Primary keys cannot contain null values.

Unique constraints can be used to ensure that the values in a column are unique, but they do not uniquely identify a row in a table. This means that a table can have multiple unique constraints. Unique constraints can contain null values.

4. What is database normalization and their types?

Database normalization is the process of organizing data in a database to minimize redundancy and dependency, while ensuring data integrity. It is a systematic way of structuring a database to improve data quality and reduce data anomalies. There are several types of database normalization, each with its own set of rules. The most common types of normalization are:

- **First normal form (1NF):** When a table has **no repeating groups**, then it says **First Normal Form (1NF)**. The repeating columns or fields present in an unauthorized table are removing from the table and put into a separate table. These tables are dependent on the parent table. The key of these tables must also be a part of the parent table so that the parent table and the derived tables can be related to each other.
- **Second normal form (2NF):** A table is in 2NF if it is in 1NF and all of its non-key attributes are fully functionally dependent on the primary key. This means **that each non-key attribute must depend on the entire primary key, not just a part of it. (increase table)**
- **Third normal form (3NF):** A table is in 3NF if it is in 2NF and there are no transitive dependencies. A transitive dependency occurs when a non-key attribute

depends on another non-key attribute. **(Eliminate unnecessary data)**

- **Boyce-Codd normal form (BCNF)/ (4th NF):** A table is in BCNF if it is in 3NF and there are no partial dependencies. A partial dependency occurs when a non-key attribute depends on only a part of the primary key. **A table is in BCNF if every functional dependency $X \rightarrow Y$, X is the super key of the table.**

Fourth normal form (4NF): Fourth Normal Form was introduced as an extension of Boyce Codd Normal Form. It eliminates in which the composite key of a record type contains two or more data items that are independent, multivalued facts of an entity.

Fifth normal form (5NF): A table is in 5NF if it is in 4NF and there are no join dependencies. A join dependency occurs when the values of two non-key attributes are related in such a way that they must always occur together in the table.

5. What is SQL?

SQL, or Structured Query Language, is a domain-specific language used in programming and designed for managing data held in a **relational database management system (RDBMS)**, or for stream processing in a relational data stream management system. SQL statements are used to perform tasks such as updating data on a database, retrieving data from a database, and creating new databases.

SQL is a standard language, which means that it is supported by most RDBMSs. This makes it a very portable language, as you can use the same SQL statements to access data on different databases.

SQL is a very versatile language, and it can be used for a wide variety of tasks. Some of the most common tasks that can be performed with SQL include:

- **Creating and managing databases:** SQL can be used to create new databases, delete existing databases, and manage the structure of databases.
- **Inserting, updating, and deleting data:** SQL can be used to insert new data into a database, update existing data in a database, and delete data from a database.
- **Querying data:** SQL can be used to query data in a database. This includes retrieving data from a database, filtering data, and sorting data.
- **Creating relationships between tables:** SQL can be used to create relationships between tables in a database. This allows you to query data from multiple tables at the same time.

6. What are the differences between DDL, DML and DCL in SQL?

DDL stands for Data Definition Language - It is used to define the structure of a database. This includes creating, altering, and dropping tables, columns, and indexes.

DML stands for Data Manipulation Language. It is used to manipulate data in a database. This includes inserting, updating, and deleting rows from tables.

DCL stands for Data Control Language. It is used to control access to a database. This includes granting and revoking permissions to users.

7. What is the difference between having and where clause?

The WHERE clause is used to filter the rows that are returned by the query. It can be used to specify conditions that the rows must meet in order to be included in the results.

The HAVING clause is used to filter the groups that are returned by the query. It can be used to specify conditions that the groups must meet in order to be included in the results.

8. What is Join?

A join is a clause that is used to combine data from two or more tables. This allows you to query data from multiple tables at the same time.

- **INNER JOIN:** This is the most common type of join. It returns all rows from the first table that have matching rows in the second table.
- **LEFT JOIN:** This returns all rows from the first table, even if there are no matching rows in the second table. The rows from the second table that do not have a match in the first table will be returned as NULL values.
- **RIGHT JOIN:** This returns all rows from the second table, even if there are no matching rows in the first table. The rows from the first table that do not have a match in the second table will be returned as NULL values.
- **FULL JOIN:** This returns all rows from both tables, whether or not there is a match in the other table. The rows from either table that do not have a match in the other table will be returned as NULL values.

9. What is a view in SQL? How to create one?

A view is a virtual table that is based on the result set of an SQL statement. A view does not physically exist in the database, but it can be used to query data from the underlying tables. **To create a view, you use the CREATE VIEW statement. The syntax for the CREATE VIEW statement is as follows:**

```
CREATE VIEW view_name AS
SELECT columns
FROM tables
WHERE conditions;
Create view customers_view as
Select cusid
```

From customer

Where name="AAA";

10. What is a Trigger and stored procedure and their difference?

A **trigger** is a special type of stored procedure that is executed automatically when a specific event occurs in a database. For example, create a trigger that is executed whenever a row is inserted into a table. The trigger could then perform some action, such as updating another table or sending an email notification.

A **stored procedure** is a set of SQL statements that are stored in the database and can be executed as a single unit. Stored procedures can be used to perform complex tasks, such as inserting, updating, and deleting data from tables. They can also be used to implement business logic and encapsulate data access.

The **main difference between triggers and stored procedures** is that triggers are executed automatically, while stored procedures are executed explicitly. This means that triggers are triggered by events, while stored procedures are called by users or other programs.

11. What is a transaction? What are ACID properties?

A transaction is a group of database operations that are treated as a single unit. Either all of the operations in the transaction are performed, or none of them are performed. This ensures that the database remains in a consistent state even if there are errors or failures.

The ACID properties are four properties that ensure the reliability of transactions. They are:

- **Atomicity:** A transaction is atomic, which means that it is either completely successful or completely unsuccessful. If any part of the transaction fails, the entire transaction is rolled back.
- **Consistency:** A transaction must maintain the consistency of the database. This means that the database must be in a valid state after the transaction is complete.
- **Isolation:** Transactions must be isolated from each other. This means that the changes made by one transaction cannot be seen by other transactions until the first transaction is complete.

- **Durability:** Once a transaction is committed, its changes are permanent. This means that the changes cannot be lost even if there is a system failure.

12. What are indexes? And what are the types of indexes?

An index is a data structure that helps a database to quickly find and retrieve rows. Indexes are created on columns that are frequently used in queries. When a query is executed, the database first searches the index to find the rows that match the query criteria. This speeds up the query execution time.

There are two main types of indexes:

- **Clustered indexes:** A clustered index is a special type of index that physically orders the rows in a table. This means that the rows in the table are stored in the same order as the index.
- **Non-clustered indexes:** A non-clustered index is an index that does not physically order the rows in a table. Instead, the index points to the rows in the table.

13. What is embedded and dynamic SQL?

Embedded SQL is a type of SQL that is embedded in a programming language. This means that the SQL statements are part of the programming language code. Embedded SQL is used to access and manipulate data in a database from within a program.

Dynamic SQL is a type of SQL that is generated at runtime. This means that the SQL statements are not known until the program is running. Dynamic SQL is used to access and manipulate data in a database in a flexible way.